VEGOVA | ELEKTROTEHNIŠKO-RAČUNALNIŠKA STROKOVNA ŠOLA IN GIMNAZIJA LJUBLJANA

LORA POZIVNIK

Strokovno poročilo

Mentor: Janez Omahen, prof.

Avtor: Jernej Leskovec, E 4. C

Ljubljana, marec 2023

POVZETEK

V tej maturitetni nalogi sem naredil napravo, ki lahko brez obstoječih omrežij doseže daljše razdalje prenosa in je kljub temu dovolj majhna, da jo lahko pospravimo v žep. Vsem tem zahtevam ustreza LoRa tehnologija. Poleg LoRe sem se odločil še za ESP32 mikrokrmilnik, ker je kompatibilen in energijsko varčnejši. Kupil sem razvojno ploščico proizvajalca LilyGo, ki ima že nameščen čip LoRa in ESP32, dodan ima pa še čip GNSS. Prispajkal sem OLED prikazovalnik, tri tipke in piezo zvočnik. Nato sem napisal programsko kodo, ki upravlja z vsemi moduli. V mislih, da ima LoRa počasen prenos podatkov, sem kodo napisal na princip »pozivnika« in s tem znižal kvantiteto prenesenih podatkov, kar direktno vpliva na porabo energije. Naslednje sem zmodeliral ohišje in ga natisnil na 3D tiskalnik. Na koncu sem pozivnika še preizkušali, dosegla sta več kot devet kilometrov dolg prenos.

KLJUČNE BESEDE: LoRa, lokalna komunikacija, prenosljivost, 3D tiskanje, ESP32, pozivnik

ABSTRACT

This project deals with developing a device, which is capable of peer-to-peer communication. The aim is to achieve the longest possible transmission distance using as little energy as possible. The device should be small enough to fit into anybody's pocket. LoRa technology is selected on the basis of parameters required and is paired with an ESP32 microcontroller. The latter is easily available, compatible, and has a low energy consumption. Firstly, a prebuilt developer kit from LilyGO, with on-board LoRa transceiver chip as well as a GNSS chip was acquired. An OLED display, three buttons, and a piezo speaker were soldered. The code, which manages all the modules was written. Considering LoRa's slow data transfer, packets are sent using "pages" to lower data transmission which directly affects time and power consumption. Next, a case for the device was designed using 3D modelling and printed on a 3D printer. The tests show a transmission length achieved is greater than nine kilometres.

KEY WORDS: LoRa, peer-to-peer, portability, 3D printing, ESP32, pager

Kazalo:

1. UVOD 7

2. METODOLOGIJA 8

2.1 BLOK SHEMA 8

2.2 ELEKTRIČNA SHEMA 9

2.3 PROGRAMSKA KODA 10

2.3.1 Vpogled 10

2.3.2 Knjižnice 11

2.3.3 Sistemske deklaracije 11

2.3.4 Globalne deklaracije in spremenljivke 11

2.3.4.1 Splošne 11

2.3.4.2 Grafični vmesnik 12

2.3.5 Začetna funkcija: setup() 12

2.3.6 Prekinitvene funkcije 12

2.3.6.1 Sprejemna prekinitvena funkcija: onRecieve() 13

2.3.6.2 Meni prekinitvena funkcija: MENUInterrupt() 13

2.3.7 Glavna zanka: loop() 13

2.3.8 GNSS razčlenitvena funkcija: LoRa_GNSS_Parser() 13

2.3.9 Funkcija za grafični vmesnik: level() 14

2.3.10 Svetilnik v sili: emergency_beacon() 14

2.3.11 Funkcija: ping() 14

2.3.12 Prikazovalnik GNSS: GNSSinfo() 15

2.3.13 Funkcija za prikaz sistemskih informaciji: sys_info() 15

2.3.14 Funkcija za nastavitve: settings() 15

2.3.15 Meni funkcija: menu() 16

2.3.16 Funkciji za pošiljanje in sprejemanje pozivov 16

2.3.16.1 Funkcija data baze vseh pozivov: pages() 16

2.3.16.2 Funkcija za pošiljanje pozivov: menuPage() ... 16
2.3.16.3 Funkcija za sprejemanje pozivov: recievedPage() ... 16
2.4 LORA NASTAVITVE ... 17
2.5 OHIŠJE ... 18
2.5.1 Načrtovanje ... 18
2.5.2 Modeliranje ... 18
2.5.3 3D tiskanje ... 18
3. PREZIKUSI DOMETA ... 19
3.1 1. PREIZKUS ... 19
3.2 2. PREIZKUS ... 20
3.3 3. PREIZKUS ... 21
3.4 4. PREIZKUS ... 22
3.5 5. PREIZKUS ... 23
4. ZAKLJUČEK ... 25
5. VIRI ... 26
6. PRILOGE ... 27

Kazalo slik

Slika 1: Blok shema pozivnika .....
Slika 2: Električna shema pozivnika 9
Slika 3: Programske knjižnice .....
Slika 4: Del sistemskih deklaraciji .....
Slika 5: Del začetne funkcije .....
Slika 6: Prekinitveni funkciji .....
Slika 7: Del glavne zanke .....
Slika 8: Del GNSS razčlenitvene funkcije .....
Slika 10: Prikazovanje GNSS podatkov, v primeru premikanja .....
Slika 9: Prikazovanje GNSS podatkov, v primeru mirovanja .....
Slika 11: Izračun smeri puščice pri GNSS prikazovalniku .....
Slika 12: OLED prikaz sistemskih informaciji .....
Slika 13: Prikaz nastavitev .....
Slika 14: OLED prikaz menija .....
Slika 15: Del data baze pozivov .....
Slika 16: Pošiljanje pozivov .....
Slika 17: Sprejemanje pozivov .....
Slika 18: 3D model trupa .....
Slika 19: Zemljevid prvega preizkusa .....
Slika 20: Teren in pot signala prvega preizkusa .....
Slika 22: Teren in pot signala drugega preizkusa .....
Slika 21: Zemljevid drugega preizkusa .....
Slika 24: Teren in pot tretjega preizkusa .....
Slika 23: Zemljevid tretjega preizkusa .....
Slika 25: Zemljevid četrtega preizkusa .....
Slika 26: Teren in pot signala četrtega preizkusa .....
Slika 27: Teren in pot signala petega preizkusa .....
Slika 28: Zemljevid petega preizkusa .....
Slika 29: Referenčna slika razdalje petega preizkusa .....
Slika 30: Končani pozivniki .....

Kazalo tabel

Tabela 1: Podatki prvega preizkusa 19
Tabela 2: Podatki drugega preizkusa 20
Tabela 3: Podatki tretjega preizkusa 21
Tabela 4: Podatki četrtega preizkusa 22
Tabela 5: Podatki petega preizkusa 23

1. UVOD

Namen mojega izdelka je bilo narediti pozivnik, ki ni odvisen od že postavljenih omrežij (npr. telefonskega) in ima sorazmerno dolg doseg. Ena glavnih uporab takega izdelka bi bilo pošiljanje klicev v sili v kriznih situacijah, kjer ni dostopa do ostalih omrežij. Naprava mora biti za opravljanje tovrstnih nalog lahka, prenosljiva in imeti dolgo baterijsko dobo. To uspe doseči tehnologiji LoRa.

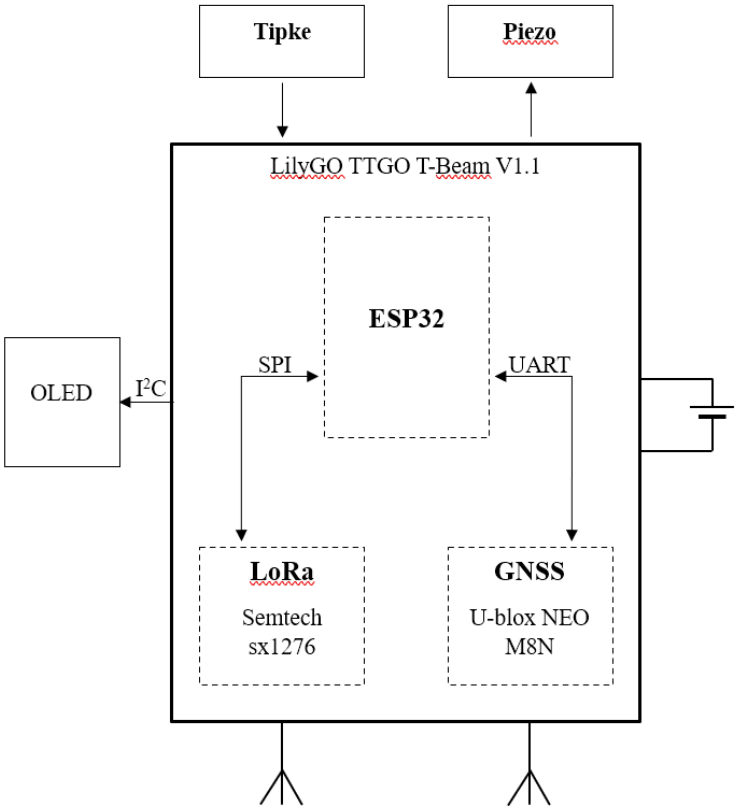
LoRa (Long Range) je tehnika radio prenosa informaciji, ki temelji na modulaciji razpršenega spektra (ang. Spread spectrum modulation), ki izvira iz CSS (Chirp spread spectrum) tehnologije. Iznašlo jo je podjetje Cycleo, ki ga je kasneje prevzelo podjetje Semtech.[1][2][3] LoRa uporablja ISM (Industrial, scientific, medical) frekvenčno območje in zato v Evropi ne potrebujem licence za oddajo.[4] LoRa ima imeniten doseg, ampak je zaradi uporabljenega frekvenčnega spektra limitirana na vidno polje. Razdalje dosega lahko pod optimalnimi pogoji dosežejo tudi do več sto kilometrov.[5]

ESP32 mikrokontroler je eden izmed najbolj kompatibilnih mikrokonotrolerjev z Arduino okoljem. Čip je precej močnejši in manjši od Arduino mikrokontrolerjev, ki smo jih uporabljali pri pouku in ima tudi več funkciji in spomina. Ima funkcijo spanja, tako ne porablja nepotrebne energije, kadar miruje.

2. METODOLOGIJA

Odločil sem se za mikrokontroler ESP32, proizvajalca Espressif Systems[6]. Zanj sem se odločil zaradi široke izbire kompatibilnih knjižnic in količine spomina. Izbral sem si LoRa modul proizvajalca Semtech. ESP32 in LoRa (Semtech SX1276) se nahajata na kupljeni razvojni ploščici podjetja LilyGo, linije TTGO, model T-Beam V1.1. Vezje dodatno vsebuje GNSS (globalni navigacijski satelitski sistem, ang. Global Navigation Satellite System) čip, podjetja U-blox, linija NEO, model M8N. Za upravljanje sem prispajkal še tri tipke in piezo zvočnik.

2.1 BLOK SHEMA



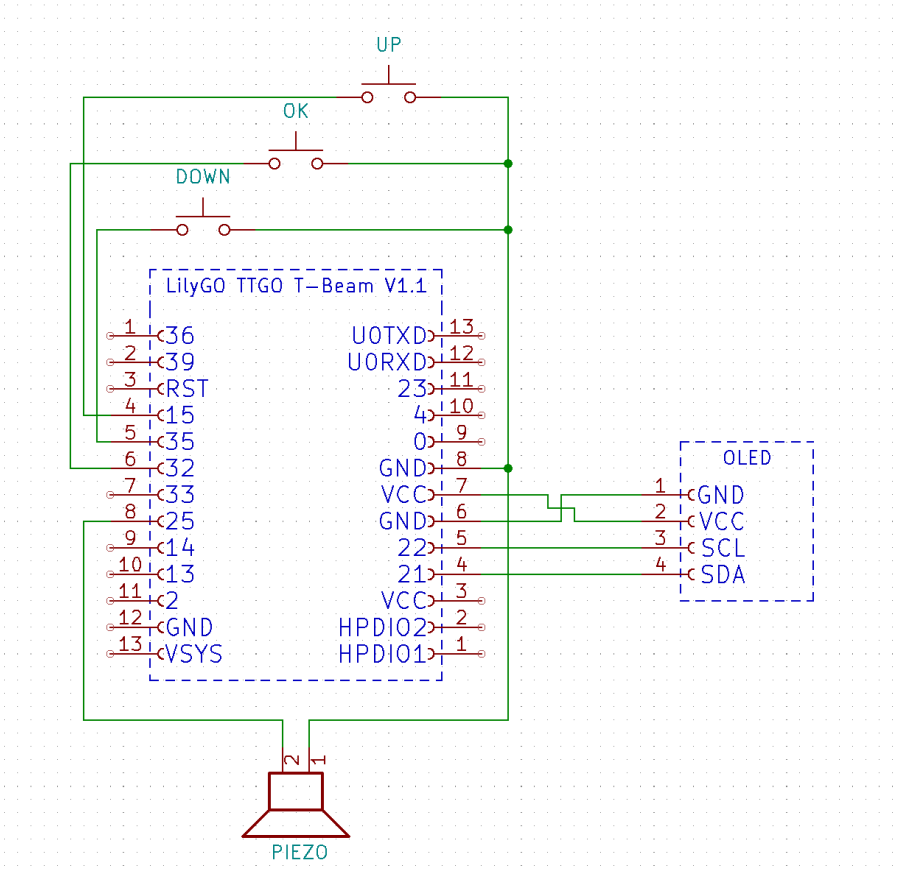
Slika 1: Blok shema pozivnika

Blok shema predstavlja samo glavne komponente:

- Za vnos podatkov o navigaciji po meniju sem uporabil tri tipke, ki so na zgornji strani ohišja in so povezane z ESP32,
- poleg tipk sem priključil tudi piezo zvočnik, ta je v ohišju, potreben je za opozarjanje o sprejetih pozivih, povezan je na ESP32,
- ESP32 komunicira z LoRa čipom preko SPI komunikacije,

- ESP32 komunicira z GNSS čipom preko zaporedne (UART) komunikacije,
- ESP32 pošilja podatke 64 x 128 OLED prikazovalniku preko I^2C,
- na vezju je še modul za upravljanje z baterijo. (več: [7])

2.2 ELEKTRIČNA SHEMA



Slika 2: Električna shema pozivnika

Opombe električne sheme:

- Tipke nimajo »pull down« upora, ker je definiran v programu in posredno vezan v samem mikrokontrolerju,
- električna shema [7] vezja LilyGO TTGO T-Beam V1.1 je priložena v prilogi,
- baterija je vezana že na samo vezje,
- povezave so narejene samo z žicami.

2.3 PROGRAMSKA KODA

Koda za ESP32 temelji na Arduino okolju, pisana v programu VSCodium (odprtokodna verzija programa VSCode) z razširitvijo Platform.io. Zaradi nizkega prenosa podatkov sem sistem pošiljanja podatkov in grafični vmesnik tako sprogramiral, da lahko na meniju izbiraš in tudi pošiljaš različne pozive. Pozivi so zasnovani na vprašanju in odgovorih (npr. na vprašanje »Where are You?« je odgovor »Heading North«). Koda ima še ostale uporabne funkcije, kot so svetilnik v sili, GNSS grafični vmesnik, »ping« program (za preizkušanje obojestranske povezave) in nastavitve.

2.3.1 Vpogled

Programsko kodo sestavljajo:

- 6 knjižnic,
- deklaracije spremenljivk in konstant,
- funkcija setup(), ki se izvrši samo enkrat na začetku programa,
- 2 prekinitveni funkciji,
- funkcija loop(), ki se izvaja kot zanka,
- 23 vmesnih funkciji, ki se izvršijo v funkciji loop().

Koda je napisana na način, da na koncu vsake izvedbe funkcije loop() ESP pade v lahko spanje[8]. Zbudimo ga z dvema prekinitvenima (ang. interrupt) funkcijama, eno funkcijo izvede pozitivni impulz na vhod 32, ki je vezan na sredinsko tipko(OK), za vstop v meni. Drugo funkcijo izvrši prekinitvena funkcija, ki je vezana na klicni izhod LoRa čipa (ang. callback)[7][9], za izpis sprejetega poziva.

Vse funkcije razen setup() in prekinitvenih funkciji se izvedejo v glavni zanki loop(). Zaradi delovanja brez zapletov so v funkcijah zelo pogosto uporabljene while zanke in millis(). Funkcije delay() sem se izogibal, ker zniža zanesljivost programa.

Določenih odsekov kode ni slikanih v dokumentu zaradi omejitve s prostorom, zato vam v prilogi prilagam celotno kodo.

(v. 10) pomeni 10 vrstica na slikah kode.

2.3.2 Knjižnice

V programu sem uporabil 6 knjižnic:

```
27  #include "axp20x.h"
28  #include <LoRa.h>
29  #include <SPI.h>
30  #include <SoftwareSerial.h>
31  #include <SparkFun_u-blox_GNSS_Arduino_Library.h>
32  #include <U8g2lib.h>
```

Slika 3: Programske knjižnice

1. Knjižnica axp20x[10] za upravljanje z nadzorom napajanja in polnjenjem baterije.
2. Knjižnica LoRa[11] za upravljanje z LoRa čipom, sprejemanjem in oddajanjem pozivov.
3. Knjižnica SPI[12], Arduino knjižnica za SPI komunikacijo, ki je uporabljena za komuniciranje z LoRa čipom.
4. Knjižnica SoftwareSerial[13] za zaporedno (UART) komunikacijo, ki se uporablja za komuniciranje z čipom GNSS.
5. Knjižnica SparkFun u-blox GNSS Arduino Library[14] za upravljanje z čipom GNSS in branje podatkov GNSS.
6. Knjižnica U8g2lib[15] za upravljanje z OLED prikazovalnikom.

2.3.3 Sistemske deklaracije

Deklaracije, ki so obvezne za delovanje sistema in komunikacijo med vsemi moduli:

- Definicija (v. 43) definira vhode, ki bodo, če bo na njih logična ena, zbudili ESP32 iz spanja. Definicija je 104000000_{16}, kar pomeni vhod 26 in 32,
- deklaracije (v. 51 do 55) označujejo nastavitve LoRa oddajnika in sprejemnika.

```
43  #define BUTTON_PIN_BITMASK 0x104000000
44
45  // Pini ki komunicirajo iz LoRa čipom
46  #define ss 18
47  #define rst 23
48  #define dio0 26
49
50  // LoRa nastavitve
51  uint8_t txPower = 20;
52  uint8_t spreadingFactor = 12;
53  long Bandwidth = 62.5E3;
54  uint8_t codingRateDenominator = 8;
55  long preambleLength = 10;
```

Slika 4: Del sistemskih deklaraciji

2.3.4 Globalne deklaracije in spremenljivke

2.3.4.1 Splošne

- Deklaracije spremenljivk (v. 74 do 77), ki so uporabljene pri sprejemanju pozivov, MSG je označen poziv, RSSI pa moč sprejema poziva,
- deklaraciji spremenljivk (v. 80, 81), ki se uporabljata kot pomnilniški celici, sta ključni za delovanje prekinitvenih funkciji,
- deklaracije spremenljivk (v. 96 do 103), ki se uporabljajo za zapisovanje in obdelavo podatkov, prejetih od čipa GNSS.

2.3.4.2 Grafični vmesnik

- Deklaracija spremenljivk (v. 111 do 146), ki so uporabljene za optimizacijo meni kode,
- definicije in deklaracije (v. 154 do 188) za risanje dveh slik.

2.3.5 Začetna funkcija: setup()

Začetna funkcija, v Arduino okolju poimenovana setup(), je odgovorna za nastavitev in inicializacijo. Izvrši se samo enkrat na začetku, ko smo mikrokontroler ali prižgali ali resetirali. Funkcija v mojem primeru opravi naslednje pomembne naloge:

- Začne komunikacijo z ostalimi moduli,
- nastavi vhode in izhode LoRa čipa, tipk in pieza,
- (v. 294 do 298) pošlje nastavitve za oddajo in sprejem LoRa čipu,
- (v. 301) ESP-ju nastavi vhode, ki ga bodo zbudili,
- (v. 302) ESP-ju nastavi vhod 32 za izvajanje prekinitvene funkcije za meni (MENUInterrupt()),
- (v. 305) nastavi vhod 26, ki bo izvršil prekinitveno funkcijo onReceive() ob sprejemu paketa,
- (v. 309) nastavi izpis čipa GNSS na samo UBX signale (Značilni za U-blox čipe) in napiše konfiguracijo v register.

```
293  // LoRa nastavitve
294  LoRa.setTxPower(txPower);
295  LoRa.setSpreadingFactor(spreadingFactor);
296  LoRa.setSignalBandwidth(Bandwidth);
297  LoRa.setCodingRate4(codingRateDenominator);
298  LoRa.setPreambleLength(preambleLength);
299
300  // Bujenje ESPja in nastavitev interupta
301  esp_sleep_enable_ext1_wakeup(BUTTON_PIN_BITMASK, ESP_EXT1_WAKEUP_ANY_HIGH);
302  attachInterrupt(digitalPinToInterrupt(32), MENUInterrupt, RISING);
303
304  // LoRa postavitev na sprejem
305  LoRa.onReceive(onReceive);          //Interrupt za sprejemanje
306  LoRa.receive();
```

Slika 5: Del začetne funkcije

2.3.6 Prekinitvene funkcije

Prekinitvene funkcije (ang. interrupt) so funkcije, ki ob določenih izpolnjenih pogojih ustavijo glavno zanko, ne glede na njeno trenutno funkcijo, izvršijo ukaze v prekinitveni funkciji in se vrnejo ter nadaljujejo glavno zanko od tam, kjer so jo prekinile.

```
244  // Interupt za sprejemanje
245  void onReceive(int packetSize) {
246      MSGstring = LoRa.readString();
247      RSSI = LoRa.packetRssi();
248      onRcounter = 1;
249      MSGlenght = MSGstring.length();
250      MSG = MSGstring.toInt();
251  }
252
253  // Interupt za zbujanje in oddajanje
254  void MENUInterrupt() {
255      counter = 1;
256  }
```

Slika 6: Prekinitveni funkciji

2.3.6.1 Sprejemna prekinitvena funkcija: onRecieve()

Funkcija, ki se izvrši, ko LoRa čip sprejme paket in izhod DIO0, ki je vezan na vhod 26, postavi na logično 1.

2.3.6.2 Meni prekinitvena funkcija: MENUInterrupt()

Prekinitvena funkcija, ki je odgovorna, da zbudi ESP iz spanja in odpre meni. Zaradi zasnove glavnega programa je dovolj, da v tej funkciji samo nastavimo »counter« pomnilno celico na 1.

2.3.7 Glavna zanka: loop()

Glavna zanka je odgovorna za pregled, katera prekinitvena funkcija je bila izvršena (preko pomnilnih celic) in odvisno od tega izvrši pravilno funkcijo. V njej je vgrajen tudi izpis sprejetih podatkov GNSS (zaradi daljšega paketa in optimizacije). Odgovorna je tudi za uspavanje GNSS in čipa ESP.

```
3513  void loop() {
3514      u8g2.setPowerSave(0);
3515      u8g2.clearDisplay();
3516      u8g2.clearBuffer();
3517      delay(500);
3518
3519      while ((counter != 0) || (onRcounter != 0)) {        //Preverimo gd
3520          if ((counter == 1) && (onRcounter == 0)) {       //Se izvrši če
3521              fdecl = 1;
3522              menu();
3523          } else if ((counter == 0) && (onRcounter == 1)) {   //V drugem
3524              reset(0, 100);
3525              fdecl = 0;
3526              if (MSGlenght > 5) {
3527                  while (whilecounter < timeouttime) {
3528                      whilecounter = millisTimeout(1000, whilecounter);
3529
3530                      if ((whilecounter % 2) == 0) {
3531                          tone(piezo, 4000, 250);
3532                      }
3533                      u8g2.clearBuffer();
3534                      u8g2.setFont(u8g2_font_7x13_mf);
3535                      u8g2.drawStr(0, 10, "RECIEVING GPS");    //Izpišemo
```

Slika 7: Del glavne zanke

Na sliki desno vidimo kodo (v. 3520- 3523), ki predstavlja logiko za pravilno klicanje funkciji, odvisno od prekinitvene funkcije, ki je bila izvedena.

- (v. 3561) Kličemo funkcijo, ki čip GNSS postavi v spanje,
- (v. 3565) kličemo funkcijo, ki čip ESP postavi v lahko spanje.

2.3.8 GNSS razčlenitvena funkcija: LoRa_GNSS_Parser()

Knjižnica za GNSS nam poda 4 podatke o lokaciji. Vsi trije podatki so zapisani kot dolgo naravno število (ang. integer), zaradi tega moramo vse štiri podatke zapisati kot en niz. To naredimo zato, da lahko pošljemo GNSS podatke v enem paketu in ne v štirih (zmanjšanje čas oddaje in posredno porabo energije). To dosežemo s funkcijo sprintf(), ki zapakira 4 naravna števila v en niz, tega pa nato pošljemo sprejemalcu. V primeru sprejemanja podatkov moramo postopek ponoviti v obratni smeri s funkcijo sscanf(), ki nam poda 4 naravna števila. Naravna števila nato spremenimo v 4 racionalna števila (float), primerna za zapis.

- Z »if« stavkom preverjamo ali smo GNSS sprejeli, ali ga bomo poslali,
- (v. 441, 442) funkcija sprintf() podatke zapakira v zbirko, katero nato pretvorimo niz,
- (v. 451, 456) pretvorimo sprejeti niz v zbirko in nato jo pretvorimo v racionalna števila.

```
440     if (recieved == 0) {
441         sprintf(buffer, "%d %d %d", latitude, longitude, altitude);
442         StringToSend = String(buffer);                    //Spreme
443         LoRa.beginPacket();
444         LoRa.print(StringToSend);
445         LoRa.endPacket(true);           //Pošljemo paket brez zahte
446         delay(4000);
447         LoRa.receive();
448         return 10;
449     }
450     if ((recieved == 1) && (MSGlenght > 5)) {
451         MSGstring.toCharArray(recievedbuffer, 32);              //Sp
452         sscanf(recievedbuffer, "%d %d %d", &Slat, &Slon, &Salt);
453
454         gpsfloatLAT = Slat / 10000000.;
455         gpsfloatLON = Slon / 10000000.;
456         gpsfloatALT = Salt / 1000.;
457
458         if (returntype == 1) {
459             return gpsfloatLAT;
```

Slika 8: Del GNSS razčlenitvene funkcije

2.3.9 Funkcija za grafični vmesnik: level()

Funkcija, ki oblikuje in izpisuje vse v meniju. Omogoča nam izbiranje in premikanje med pozivi in ostalimi funkcionalnostmi programa. Upravljamo jo s tipkami - leva tipka nas premika dol, desna gor in sredinska naprej. Funkcija je ključna za optimizacijo kode, saj jo skrajša za najmanj 5000 vrstic.

- Najprej vsem nizom, ki jih bomo zapisali, določimo dolžino (za sredinsko poravnavo),
- nato izpišemo prvo stran menija,
- potem konstantno preverjamo, ali je tipka pritisnjena, in če je, se kazalec premakne,
- če imamo več opcij, se nadaljujejo še na drugi in tretji strani.

2.3.10 Svetilnik v sili: emergency_beacon()

Svetilnik v sili oddaja signal vsakih 10 sekund in istočasno išče GNSS satelite. Ko enkrat čip fiksira lokacijo, jo začne oddajati. Funkcija, ki to sprejema (emergency_rec()), izpiše sprejemanje in lokacijo GNSS na OLED prikazovalnik.

2.3.11 Funkcija: ping()

Funkcija za preizkušanje dvostranske povezave med pozivnikoma, najprej pošlje paket in nato čaka drugi pozivnik, ki paket sprejme in odda identičnega nazaj. Če je ping uspešen, funkcija to izpiše na prikazovalnik poleg moči sprejema, v nasprotnem primeru pa izpiše, da ni uspešen.

2.3.12 Prikazovalnik GNSS: GNSSinfo()

Funkcija prikazovalnik GNSS vklopi sprejemnik GNSS in čaka na njegove podatke. Ko se GNSS fiksira, funkcija na OLED prikazovalnik izpiše sprejete podatke. Izpisani podatki so zemljepisna širina, zemljepisna dolžina, nadmorska višina, napaka v lokaciji, število satelitov, s katerih sprejema podatke, hitrost. Smer hitrosti je prikazana grafično. Izpiše tudi čas in datum, ki sta shranjena v notranji uri čipa GNSS.

Slika 9: Prikazovanje GNSS podatkov, v primeru premikanja

Slika 10: Prikazovanje GNSS podatkov, v primeru mirovanja

- (v 2261 do 2270) Enačbe za grafičen prikaz smeri hitrosti.

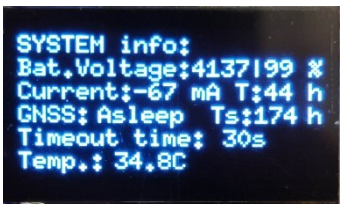
```
2261    angle_rad = (myGNSS.getHeading() * 10 ^ 5) * M_PI / 180;
2262    x1 = (16 * sin(angle_rad)) + 100;
2263    y1 = (16 * cos(angle_rad)) + 45;
2264    x2 = (-16 * sin(angle_rad)) + 100;
2265    y2 = (-16 * cos(angle_rad)) + 45;
2266
2267    y3 = (13 * cos(angle_rad - (15 * M_PI / 180))) + 45;
2268    y4 = (13 * cos(angle_rad + (15 * M_PI / 180))) + 45;
2269    x3 = (13 * sin(angle_rad - (15 * M_PI / 180))) + 100;
2270    x4 = (13 * sin(angle_rad + (15 * M_PI / 180))) + 100;
```

Slika 11: Izračun smeri puščice pri GNSS prikazovalniku

2.3.13 Funkcija za prikaz sistemskih informaciji: sys_info()

Prikazane sistemske informacije se predvsem osredotočajo na baterijo in polnjenje. Funkcija prikazuje trenutno napetost baterije, njen odstotek napolnjenosti, tok, s katerim se polni oziroma prazni, koliko časa bo baterija še polna oziroma koliko časa se bo še polnila.

- Napetost na bateriji je podana v milivoltih,
- »T:« prikazuje čas, v katerem se baterija izprazni ob trenutnih pogojih uporabe,
- »Ts:« prikazuje čas, v katerem bi se baterija izpraznila v stanju spanja.

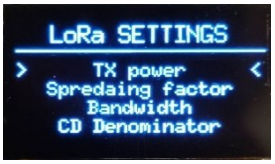


Slika 12: OLED prikaz sistemskih informaciji

2.3.14 Funkcija za nastavitve: settings()

Nastavitve najdemo v meniju in nam omogočajo spreminjanje:

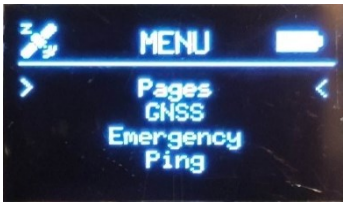
- GNSS nastavitev,
- LoRa nastavitev,
- čas izklopa,
- meni za izklop.



Slika 13: Prikaz nastavitev

2.3.15 Meni funkcija: menu()

Meni funkcija služi izbiranju med pozivi in ostalimi funkcijami, ki so naštete zgoraj. Funkcija level() prikazuje možnosti na OLED prikazovalniku. Zaradi zasnove level() funkcije mi spreminjamo samo deklaracije nizov, ki jih hočemo prikazati. Zbiranje deluje preko »switch-case« stavka. Funkcija meni na prvi strani grafično prikazuje tudi povezavo GNSS in vrednosti baterije.



Slika 14: OLED prikaz menija

2.3.16 Funkciji za pošiljanje in sprejemanje pozivov

2.3.16.1 Funkcija data baze vseh pozivov: pages()

Funkciji za pošiljanje in sprejemanje za delovanje potrebujeta data bazo pozivov. Vsakemu pozivu pripada ena številka, ki jo nato pošljemo sprejemalcu, ta mora za pravilno delovanje imeti isto data bazo. Data baza in grafični vmesnik v meniju sta lahko prilagodljiva, pozivov je lahko tudi več tisoč. Data baza je zapisana v obliki »switch case« stavka.

```
997     case 3:
998         page = "Where are you?";
999         functionswitch(Switch);
1000        break;
1001    case 4:
1002        page = "GPSn Smwhr N";
1003        functionswitch(Switch);
1004        break;
1005    case 5:
1006        page = "GPSn Smwhr NE";
1007        functionswitch(Switch);
1008        break;
1009    case 6:
1010        page = "GPSn Smwhr E";
```

Slika 15: Del data baze pozivov

2.3.16.2 Funkcija za pošiljanje pozivov: menuPage()

Funkcija naprej izpiše poziv, ki ga hočemo poslati in ga pošlje, če pritisnemo tipko.

Slika 16: Pošiljanje pozivov

2.3.16.3 Funkcija za sprejemanje pozivov: recievedPage()

Funkcija na OLED prikazovalniku izpiše poziv, ki smo ga sprejeli in moč sprejema.

Slika 17: Sprejemanje pozivov

2.4 LORA NASTAVITVE

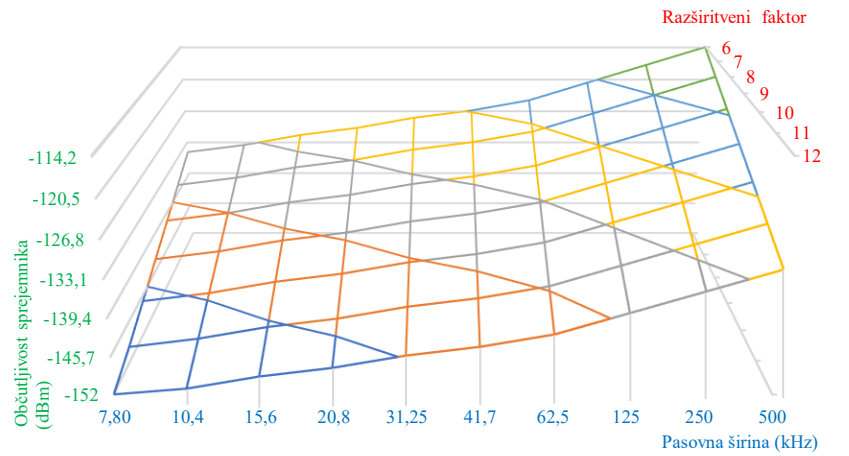
Do nastavitev in kako te vplivajo na dolžino dosega, čas in porabo sem prišel s pomočjo programa od Semtecha[16], ki sem ga obratno inženiril (ang. reverse engineering) v MS Excelu. To sem naredil zato, ker nikjer nisem našel enostavnih oziroma razumljivih enačb za nastavitve.

Nastavitve modulacije LoRa so ključne za doseganje dolgih razdalj prenosa[17]. Nastavitve, ki pridejo v poštev pri doseganju daljših razdalj, so:

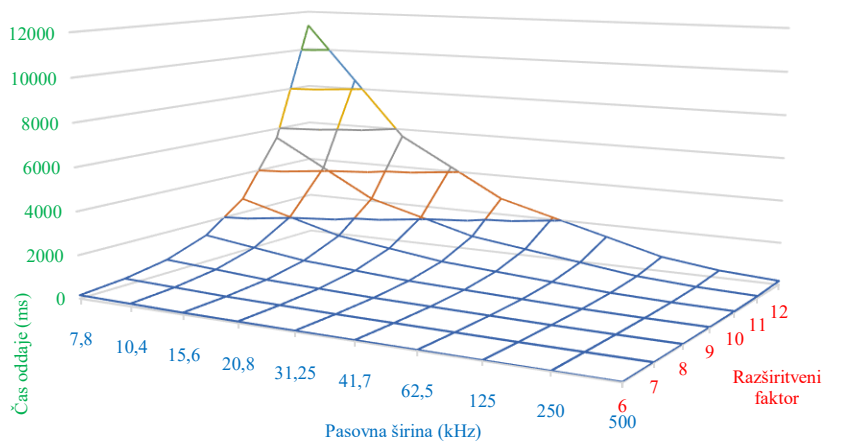
- moč oddaje (dB) določa, s kakšno močjo signal oddamo (polna moč je odvisna od antene),
- razširitveni faktor določa čas trajanja čivka,
- pasovna širina (kHz) določa širino frekvenčnega pasu komunikacije.

Občutljivost sprejemnika neposredno vpliva na doseg signala. Nižja, kot je, daljše razdalje dosegamo in več ovir lahko premagamo. Poleg občutljivosti sta pomembni tudi moč oddaje in vrsta antene. Na grafu (1) vidimo, da večji razširitveni faktor in nižja pasovna dolžina pri istem ojačenju pripomoreta k najnižji občutljivosti.

Graf 1: OBČUTLJIVOST SPREJEMNIKA (20dB ojačanje)



Graf 2: ČAS ODDAJE PAKETA (2byta paketa in 10 bitov pred paketa)

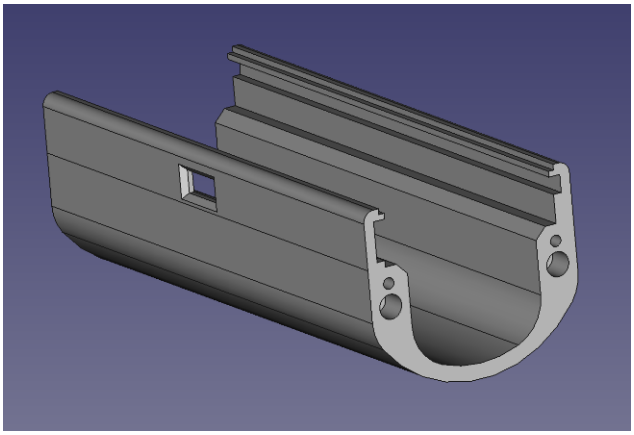


Čas oddaje (odvisen tudi od dolžine paketa) je poleg občutljivosti sprejemnika zelo pomemben, ker je skupaj z močjo oddaje odgovoren za porabljeno energijo pri oddaji paketa. Iz grafa (2) vidimo, da daljši doseg pomeni daljši čas oddaje in s tem tudi premosorazmerno več porabljene energije.

V svoji konfiguraciji sem uporabil razširitveni faktor 12 v paru s pasovno širino 62,5 kHz (zaradi neznanega vzroka, so se mi paketi nižjih pasovnih širin popačili, zato mi ni uspelo dobiti najboljših rezultatov). Oddajal sem z močjo 20 dB (0,1 W). Ostale nastavitve sem imel nastavljene na CR imenovalec 4/8 (pripomore v primeru motenj) in dolžino pred paketa 10 bitov. Dolžina paketa je 2 bajta, razen v primeru pošiljanja podatkov GNSS.

2.5 OHIŠJE

Ohišje sem načrtoval in zrisal v programu FreeCAD in nato 3D natisnil na 3D tiskalniku Prusa. Na sprednjo stran sem nalepil 2 mm debelo akrilno steklo, saj sem menil, da bo to izboljšalo videz. Vsi modeli so priloženi.



Slika 18: 3D model trupa

2.5.1 Načrtovanje

Najprej sem s kljunastim merilom izmeril vse mere in jih zapisal na papir, si na hitro skiciral in začel modelirati. Programske skice sem načrtoval precej skromno, zaradi možnosti naknadnega popravljanja.

2.5.2 Modeliranje

Modelirati sem začel trup, nato pa še stranska pokrovčka. Imel sem več težav, ker sem prvič uporabljal program in ga sproti spoznaval. Ko sem končal modeliranje, sem ohišje poskusil 3D natisniti.

2.5.3 3D tiskanje

3D tiskanje celega ohišja je v povprečju trajalo od 3 do 4 ure. Ko se je natisnilo prvo ohišje, so se začele pojavljati težave. Spoznal sem, da deli ne sedejo skupaj, tako kot sem načrtoval. Po spremenjenem modelu sem ohišje ponovno natisnil, ampak se spet ni sestavilo po načrtih. Tretjič sem se odločil, da bom začel od začetka. Po skoraj idealnem poskusnem tisku sem naredil samo še malo sprememb in dal tiskati. Čeprav sem prezrl majhno napako, sem natisnil še ostali dve ohišji, in napako kasneje popravil s skalpelom. Natisnjena ohišja so bila iz belega filamenta, zato sem se odločil, da bom dva od njiju prebarval z različnimi barvami. Pred barvanjem sem jih zbrusil, saj sem hotel prikriti sledi 3D tiskanja. Dve ohišji sem prebarval in medtem izrezal akrilna stekla, ki sem jih prilepil na sprednji del ohišja. Vrezal sem še vijačnice in vse skupaj sestavil.

3. PREZIKUSI DOMETA

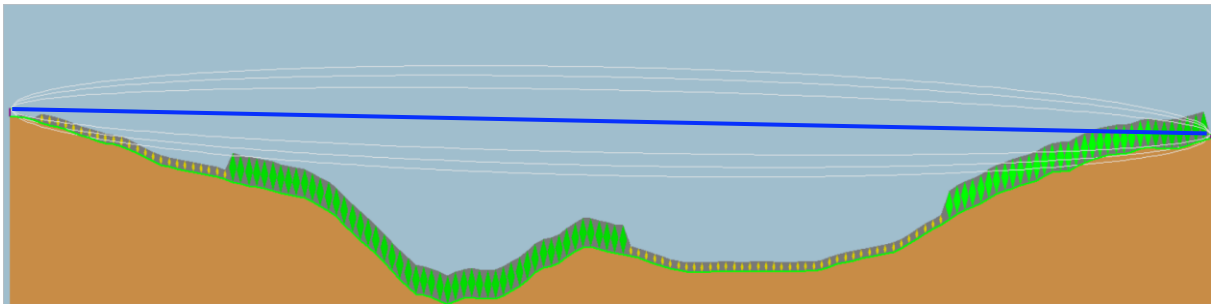
Po uspešnem sestavljanju in programiranju sem opravil 5 preizkusov. Preizkušal sem doseg pozivnika v vidni liniji. Enega sem postavil na okensko polico in mu vključil funkcijo svetilnika v sili. Z drugim sem preizkušal maksimalno razdaljo dosega. Izvajal sem jih v sorazmerno gričevnatem terenu, kjer je bil kar izziv najti vidno linijo z oddajnikom. Rezultati so me presenetili, saj nisem imel idealne kombinacije nastavitev. Preizkusi so bili izvedeni na isti dan, med točkami sem se premikal z avtomobilom.

3.1 1. PREIZKUS

	ODDAJNIK	TEST1
Zemljepisna širina	46,031483	46,019708
Zemljepisna dolžina	14,05466	14,044941
Nadmorska višina	783m	772m
Višina antene	4m	2m
Moč oddaje	20dB	
Pasovna širina	62,5 kHz	62,5 kHz
Razširitveni faktor	12	12
DOLŽINA SPREJEMA:		**1509m**
MOČ SPREJEMA: (RSSI)		-116 dBm

Tabela 1: Podatki prvega preizkusa

Slika 19: Zemljevid prvega preizkusa



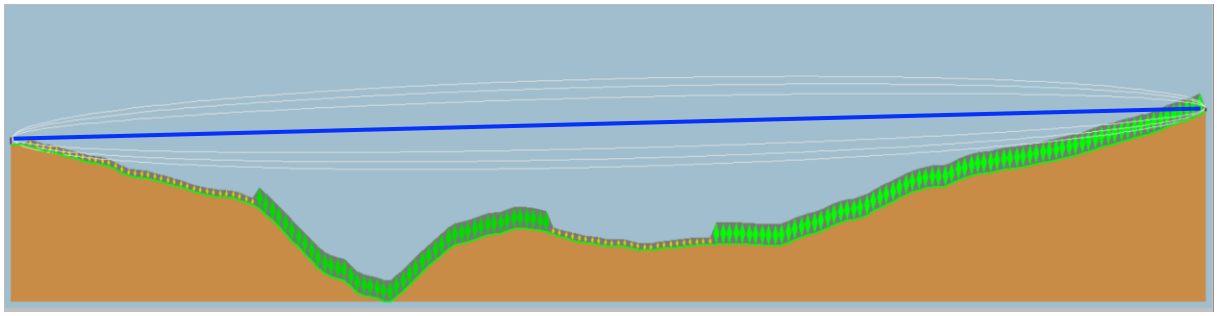
Slika 20: Teren in pot signala prvega preizkusa

3.2 2. PREIZKUS

	ODDAJNIK	TEST2
Zemljepisna širina	46,031483	46,016199
Zemljepisna dolžina	14,05466	14,046049
Nadmorska višina	783m	806m
Višina antene	4m	2m
Moč oddaje	20dB	
Pasovna širina	62,5 kHz	62,5 kHz
Razširitveni faktor	12	12
DOLŽINA SPREJEMA:		**1825m**
MOČ SPREJEMA: (RSSI)		-113 dBm

Tabela 2: Podatki drugega preizkusa

Slika 22: Zemljevid drugega preizkusa

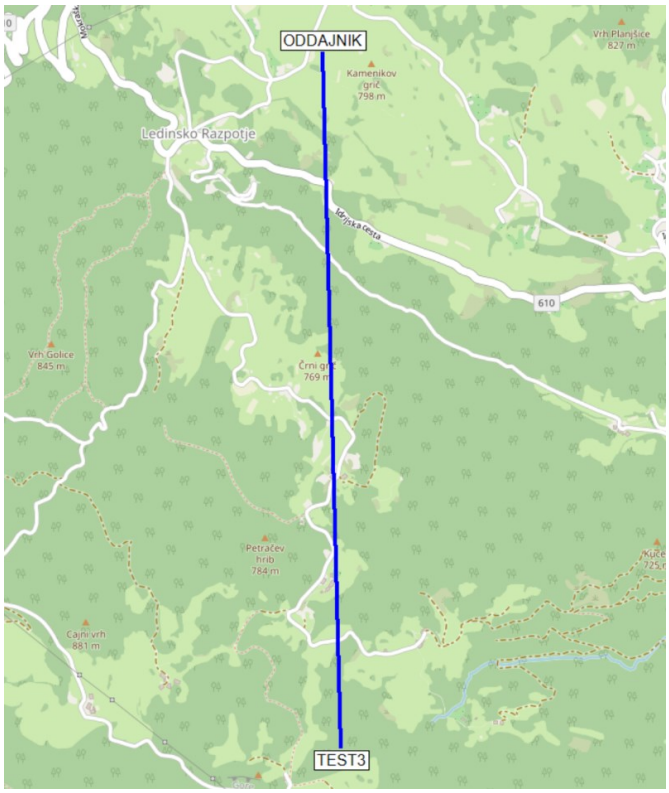


Slika 21: Teren in pot signala drugega preizkusa

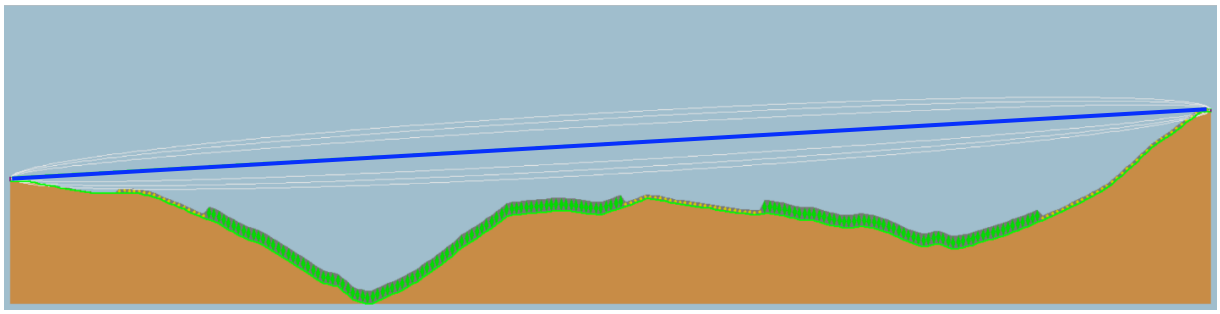
3.3 3. PREIZKUS

	ODDAJNIK	TEST3
Zemljepisna širina	46,031483	46,009801
Zemljepisna dolžina	14,05466	14,055484
Nadmorska višina	783m	864m
Višina antene	4m	2m
Moč oddaje	20dB	
Pasovna širina	62,5 kHz	62,5 kHz
Razširitveni faktor	12	12
DOLŽINA SPREJEMA:		**2412m**
MOČ SPREJEMA: (RSSI)		-114 dBm

Tabela 3: Podatki tretjega preizkusa



Slika 24: Zemljevid tretjega preizkusa

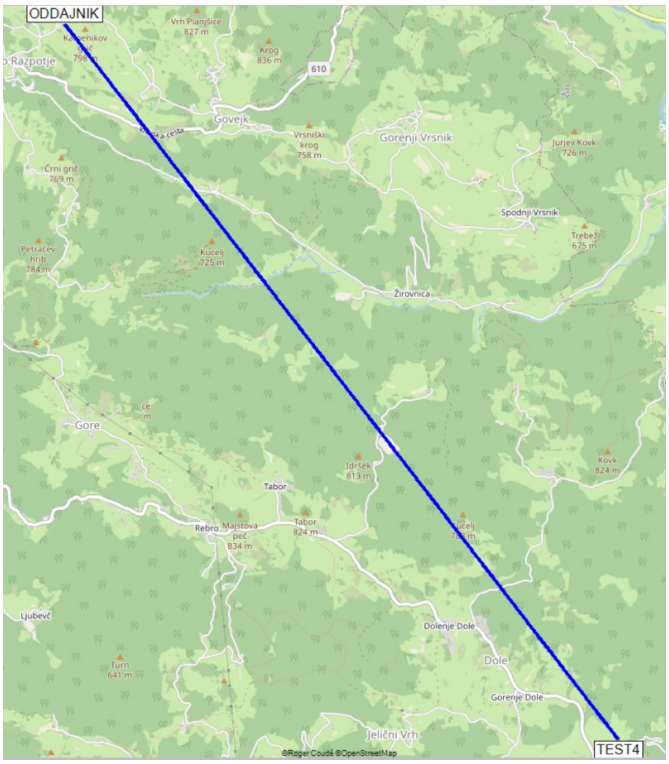


Slika 23: Teren in pot tretjega preizkusa

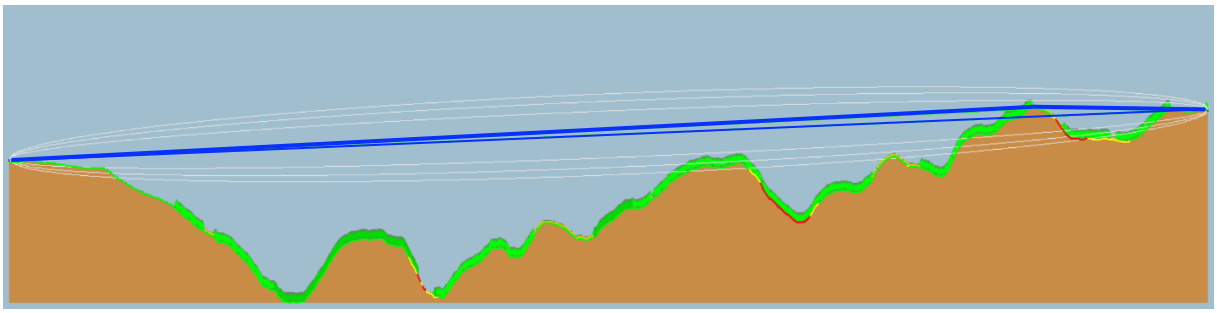
3.4 4. PREIZKUS

	ODDAJNIK	TEST4
Zemljepisna širina	46,031483	45,981353
Zemljepisna dolžina	14,05466	14,110538
Nadmorska višina	783m	859m
Višina antene	4m	2m
Moč oddaje	20dB	
Pasovna širina	62,5 kHz	62,5 kHz
Razširitveni faktor	12	12
DOLŽINA SPREJEMA:		**7050m**
MOČ SPREJEMA: (RSSI)		-131 dBm

Tabela 4: Podatki četrtega preizkusa



Slika 25: Zemljevid četrtega preizkusa



Slika 26: Teren in pot signala četrtega preizkusa

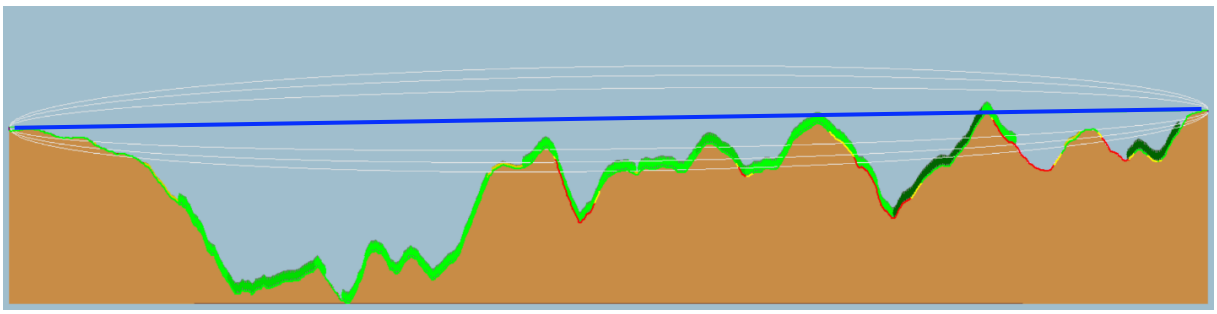
3.5 5. PREIZKUS

	Zemljepisna širina	Zemljepisna dolžina	Nadmorska višina	Višina antene	Moč oddaje	Pasovna širina	Razširitveni faktor
ODDAJNIK	46,031483	14,05466	783m	4m	20dB	62,5 kHz	12
TEST5	45,970606	14,135921	808m	2m		62,5 kHz	12
DOLŽINA SPREJEMA:		**9231m**	MOČ SPREJEMA: (RSSI)				-132

Tabela 5: Podatki petega preizkusa

Slika 29: Referenčna slika razdalje petega preizkusa

Slika 28: Zemljevid petega preizkusa



Slika 27: Teren in pot signala petega preizkusa

Vse preizkuse sem opravljal na isti lokaciji, zato so zemljepisna širina in dolžina ter nadmorska višina enake. Prav tako se niso spreminjali parametri, kot so širina antene, moč oddaje in pasovna širina. Pri prvem preizkusu je bila razdalja med sprejemnikom in oddajnikom najkrajša, in sicer 1509 m, moč sprejema pa pri -116 dBm najboljša. Pri drugem preizkusu je bila pri 1825 m razdalje moč sprejema -113 dBm, pri tretjem preizkusu pa pri razdalji 2412 m, moč sprejema -114 dBm. Ti trije preizkusi so pokazali najboljše rezultate, kar sem tudi pričakoval, saj razdalja ni bila prevelika in je bil oddajnik v vidnem polju sprejemnika. Četrti preizkus za razliko od prejšnjih treh ni bil v vidnem polju, tudi razdalja je bila večja, in sicer 7050 m. Rezultat me je presenetil, ker oddajnik ni bil v vidnem polju, zato sprejema signala nisem pričakoval. Je pa v skladu s tem bila moč signala manjša, in sicer -131 dBm. Sprejem petega preizkusa tudi ni bil pričakovan, ker nisem uporabljal idealnih LoRa nastavitev in sem pričakoval slabše rezultate na daljše razdalje, v tem primeru 9231 m. Moč signala je bila tukaj najnižja, -132 dBm, ampak vseeno ostaja še 10 dBm do teoretične meje sprejema.

4. ZAKLJUČEK

Cilj maturitetne naloge je bil dokončan izdelek, ampak sem od nje odnesel precej več. Bolj podrobneje sem spoznal tehnologijo LoRa in ESP32 krmilnik, kako moduli med sabo komunicirajo in delovanje sprejemnika GNSS. Najbolj pa sem izboljšal svoje znanje C++ programskega jezika in potrpežljivost, ki je potrebna za reševanje programskih izzivov. Uspelo mi je narediti delujoč izdelek, ki z močjo 0.1 W dosega tudi do 10 km dosega. Ima grafični vmesnik in funkcionalno kodo z več kot stotimi pozivi. Pozivnik ima na sprejemu porabo pod 16 mA, kar pomeni, da zdrži s trenutnimi baterijami en teden brez polnjenja.

Kljub temu da ima naprava dolg doseg, nisem prišel niti blizu polne sposobnosti LoRa tehnologije. Prvič, uporabljal sem nastavitve, ki niso bile idealne za doseganje najdaljših razdalji komunikacije, tudi antena, ki sem jo uporabljal, je bila dolga samo približno 5 cm, za daljše povezave bi moral uporabljati 15 ali več centimetrsko anteno.

Izboljšava, ki mi je ni uspelo doseči, je vodotesno ohišje, podcenjeval sem že izdelavo normalnega ohišja. Izdelek bi lahko bil nadgrajen na več možnih načinov, eden izmed njih je izboljšati kodo, predvsem odpravo nepravilnosti in napak. Program se večkrat zatakne in ga je treba resetirati, vzrok hroščev je trenutno še neznan. Naslednja izboljšava bi lahko bila povezava na satelite preko tehnologije TinyGS[18], in tako povečanje teoretičnega dosega komunikatorja do več tisoč kilometrov. Z zamenjavo ESP32 mikrokontrolerja z mikrokontrolerjem STM bi pridobili nižjo porabo in večjo učinkovitost. Število načinov, kako izboljšati ta projekt, limitira proti neskončnosti, saj je tehnologija uporabe nizke porabe za dolge komunikacije zelo obetavna. Tudi sam se bom še v prihodnje še ukvarjal s svojim projektom in njegovo nadgradnjo.

Slika 30: Končani pozivniki

5. VIRI

[1] https://www.semtech.com/lora/what-is-lora (23. 3. 2023)

[2] https://web.archive.org/web/20190718200516/https://www.semtech.com/uploads/documents/an1200.22.pdf (23. 3. 2023)

[3] https://www.design-reuse.com/news/28706/semtech-cycleo-acquisition.html (23. 3. 2023)

[4] 2021 LoRa Alliance https://lora-alliance.org/wp-content/uploads/2021/05/RP-2-1.0.3.pdf (24. 3. 2023)

[5] LoRa Alliance https://lora-alliance.org/lorawan-news/lorawanr-distance-world-record-broken-twice-766-km-476-miles-using-25mw-transmission/ (24. 3. 2023)

[6] 2023 Espressif Systems https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf (24. 3. 2023)

[7] Xinyuan-LilyGO https://github.com/Xinyuan-LilyGO/LilyGo-LoRa-Series/blob/master/schematic/LilyGo_TBeam_V1.1.pdf (25. 3. 2023)

[8] 2016 – 2023 Espressif Systems https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/sleep_modes.html (25. 3. 2023)

[9] Sandeepmisty https://github.com/sandeepmistry/arduino-LoRa/blob/master/examples/LoRaReceiverCallback/LoRaReceiverCallback.ino (25. 3. 2023)

[10] Lyusupov https://github.com/lyusupov/SoftRF/tree/master/software/firmware/source/libraries/AXP202X_Library (25. 3. 2023)

[11] Sandeepmisty https://github.com/sandeepmistry/arduino-LoRa (25. 3. 2023)

[12] Arduino https://github.com/arduino/ArduinoCore-avr/tree/master/libraries/SPI (25. 3. 2023)

[13] Plerup https://github.com/plerup/espsoftwareserial (25. 3. 2023)

[14] Sparkfun https://github.com/sparkfun/SparkFun_u-blox_GNSS_Arduino_Library (25. 3. 2023)

[15] Olikraus https://github.com/olikraus/u8g2 (25. 3. 2023)

[16] https://sx1272-lora-calculator.software.informer.com/download/?cadc549 (25. 3. 2023)

[17] Martin Bor, Utz Roedig. 2017. *School of Computing & Communications Lancaster University*. LoRa Transmission Parameter Selection. Dostopno prek: https://eprints.lancs.ac.uk/id/eprint/85515/4/lora_tps_r1342.pdf (28. 3. 2023)

[18] 2023 TinyGS https://tinygs.com/ (28. 3. 2023)

6. PRILOGE

1. Priloga: MATURA_POROČILO_Jernej_Leskovec_E4C_PROGRAMSKAKODA.cpp
2. Priloga: LilyGo_TBeam_V1.1.pdf
3. Priloga: MATURA_POROČILO_Jernej_Leskovec_E4C_3DMODEL_OHIŠJE.FCStd
4. Priloga: MATURA_POROČILO_Jernej_Leskovec_E4C_3DMODEL_POKROVCEK2.FCStd
5. Priloga: MATURA_POROČILO_Jernej_Leskovec_E4C_3DMODEL_POKROVCEK1.FCStd

Izjava o avtorstvu

Izjavljam, da je strokovno poročilo Lora pozivnik v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

Ljubljana, 30. 3. 2023 *Jernej Leskovec,*